Chapter 14 Indexing

- *Basic Concepts*
- *Ordered Indices:* Primary index, Secondary index
- *B^+-Tree Index*
- *Hash Indices*
- *Index Definition in SQL*

Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
- **Search Key** – An attribute or set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices:** search keys are stored in sorted order
  - **Hash indices:** search keys are distributed uniformly across "buckets" using a "hash function".

Index Evaluation Metrics

- Access types supported efficiently.
  - records with a specified value in the attribute
  - or records with an attribute value falling in a specified range of values.
- Access time
- Insertion time
- Deletion time
- Space overhead

Ordered Indices

In an **ordered index,** index entries are stored sorted on the search key value:

- **Primary index（主索引）:** in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
  - Also called **clustering index（聚集索引）**
  - The search key of a primary index is usually but not necessarily the primary key.

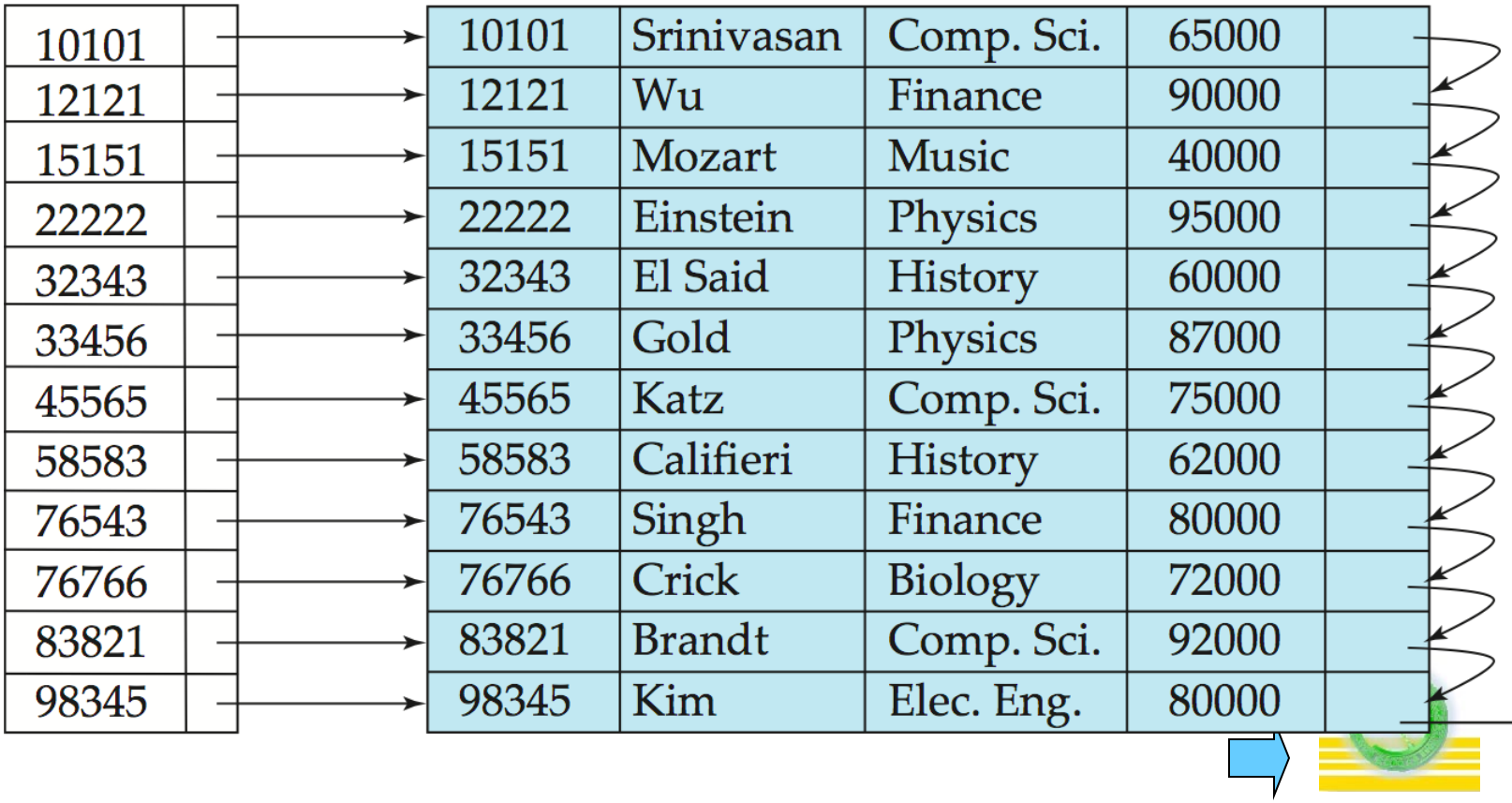
Index-sequential file: ordered sequential file with a primary index.

- **Secondary index（辅助索引）**: an index whose search key specifies an order different from the sequential order of the file. Also called non-clustering index**.**

Dense Indices(稠密索引)

■ Dense index — Index record appears for every search-key value in the file.

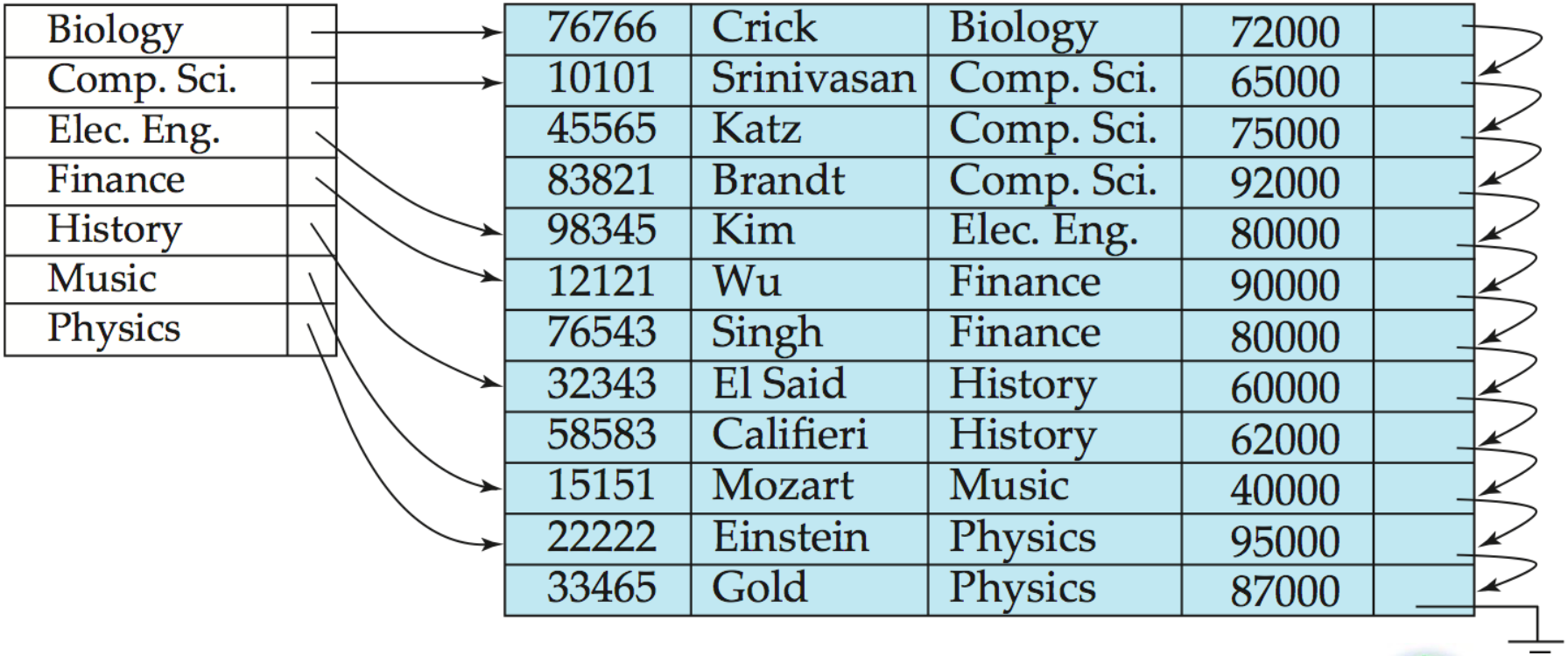
■E.g. index on *ID* attribute of *instructor* relation



10101	10101	Srinivasan	Comp. Sci.	65000
12121	12121	Wu	Finance	90000
15151	15151	Mozart	Music	40000
22222	22222	Einstein	Physics	95000
32343	32343	El Said	History	60000
33456	33456	Gold	Physics	87000
45565	45565	Katz	Comp. Sci.	75000
58583	58583	Califieri	History	62000
76543	76543	Singh	Finance	80000
76766	76766	Crick	Biology	72000
83821	83821	Brandt	Comp. Sci.	92000
98345	98345	Kim	Elec. Eng.	80000

Dense Indices

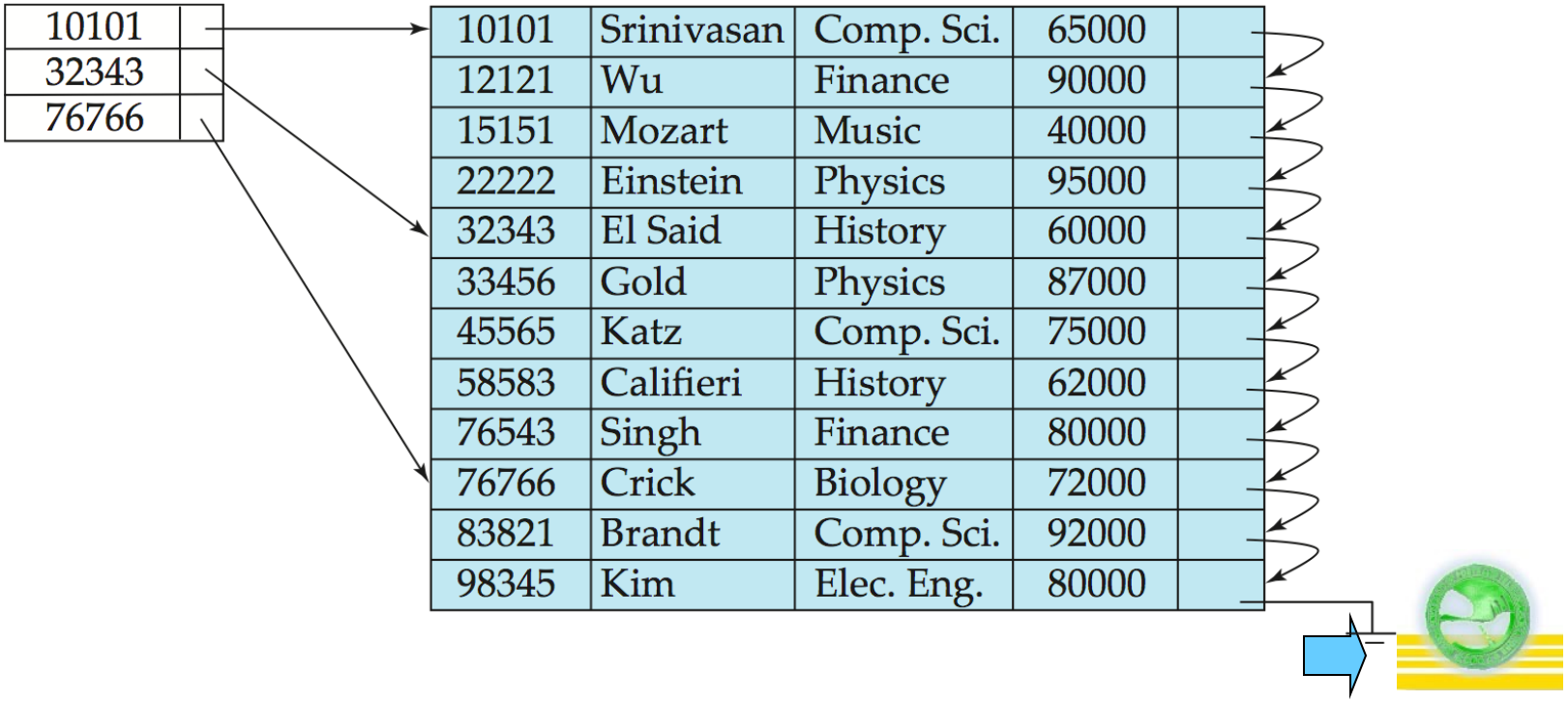
- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*



Sparse Indices（稀疏索引）

■ Sparse Index: contains index records for only some search-key values.

➢ Applicable when records are sequentially ordered on search-key



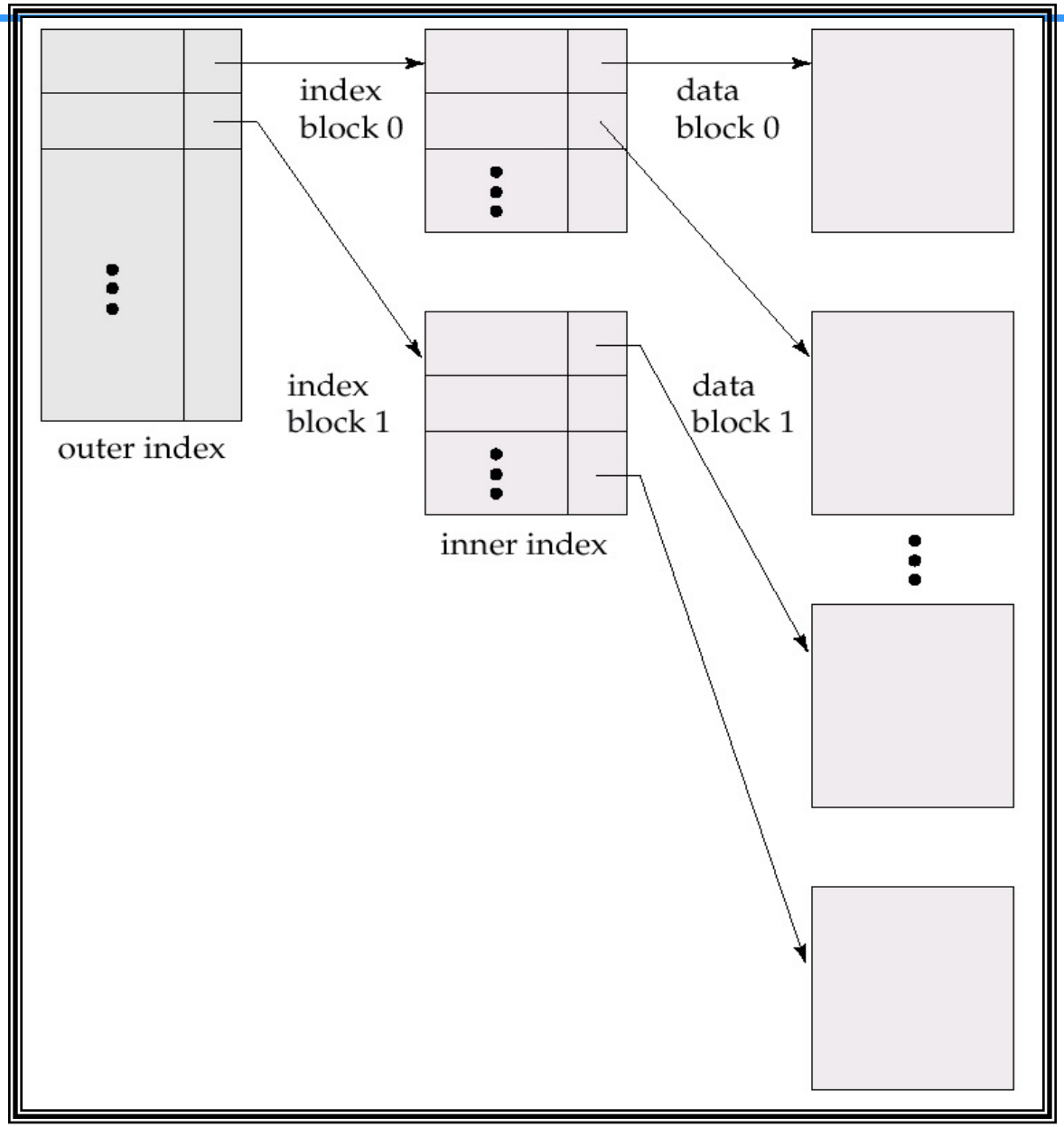
Sparse Indices(2)

- To locate a record with search-key value K we:
  - Find index record with largest search-key value $< K$
  - Search file sequentially starting at the record to which the index record points
- Less space and less maintenance overhead for insertions and deletions.
- Generally slower than dense index for locating records.
- Good tradeoff: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.

Multilevel Index

- If primary index does not fit in memory, access becomes expensive.
- To reduce number of disk accesses to index records, treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - outer index – a sparse index of primary index
  - inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

Example


index
block 0
data
block 0
index
block 1
data
block 1
outer index
inner index

Index Update: Deletion

- If deleted record was the only record in the file with its particular search-key value, the search-key is deleted from the index also.
- Single-level index deletion:
  - Dense indices –

    Delete the pointers(stores pointers to all records);

    Update the pointer (stores pointers to the first record);
  - Sparse indices – Nothing done;

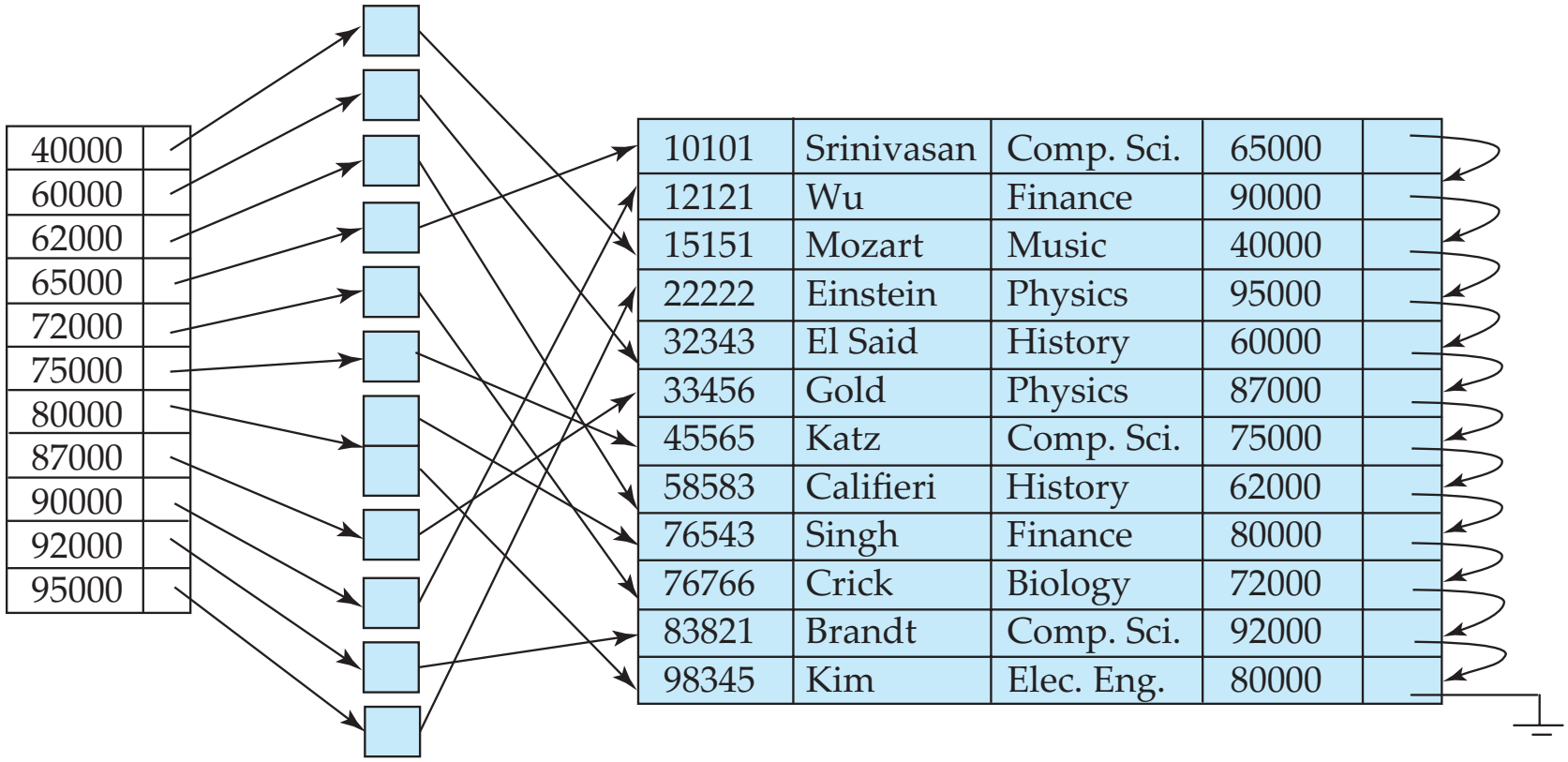
    replace with next search-key value;

    updates the pointer;
- Multilevel deletion algorithms are simple extensions of the single-level algorithms.

Index Update: Insertion

- Single-level index insertion:
  - Perform a lookup using the search-key value appearing in the record to be inserted.
  - Dense indices –insert index/add pointer/update pointer
  - Sparse indices – no change / insert index / update pointer
- Multilevel insertion algorithms are simple extensions of the single-level algorithms

Secondary Indices



Secondary index on *salary* field of *instructor*

Primary and Secondary Indices

- Secondary indices have to be dense.
- Indices offer substantial benefits when searching for records.
- When a file is modified, every index on the file must be updated, Updating indices imposes overhead on database modification.
- Sequential scan using primary index is efficient, but a sequential scan using a secondary index is expensive

  each record access may fetch a new block from disk

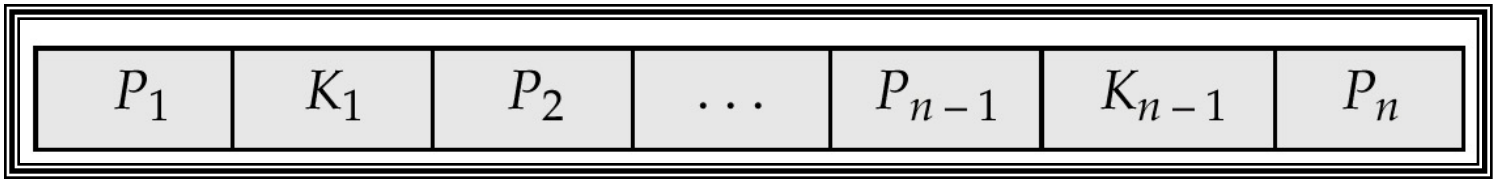
B^+-Tree Index Files

A B^+-tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $[n/2]$ and n children.
- A leaf node has between $[(n-1)/2]$ and $n-1$ values
- Special cases:
  - If the root is not a leaf, it has at least 2 children.
  - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.

B^+- Tree Node Structure

Typical node:



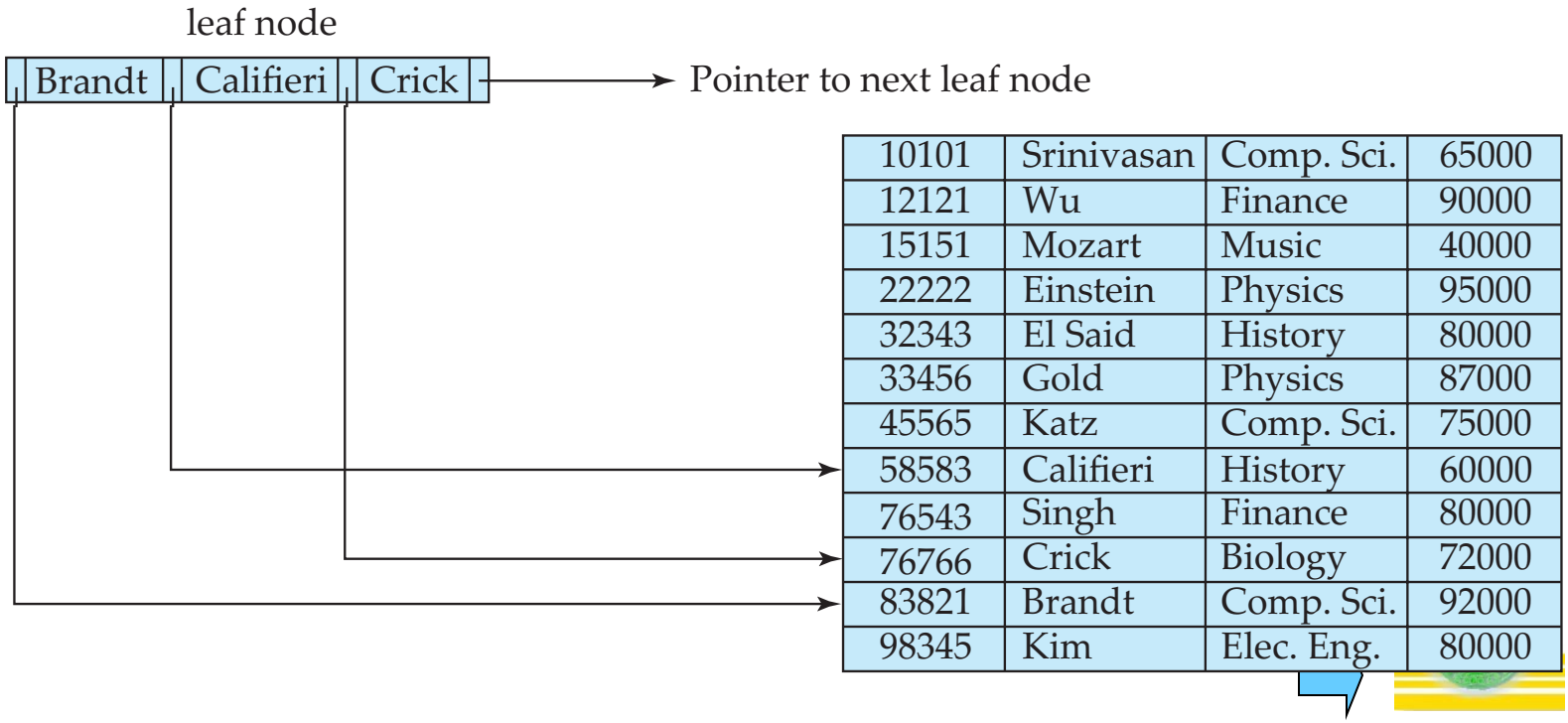
- K_i are the search-key values
- P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).

The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

Leaf Nodes in B+-Trees

- For $i = 1, 2, \ldots, n-1$, pointer P_i points to a file record with search-key value K_i,
- If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than or equal to L_j's search-key values
- P_n points to next leaf node in search-key order



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

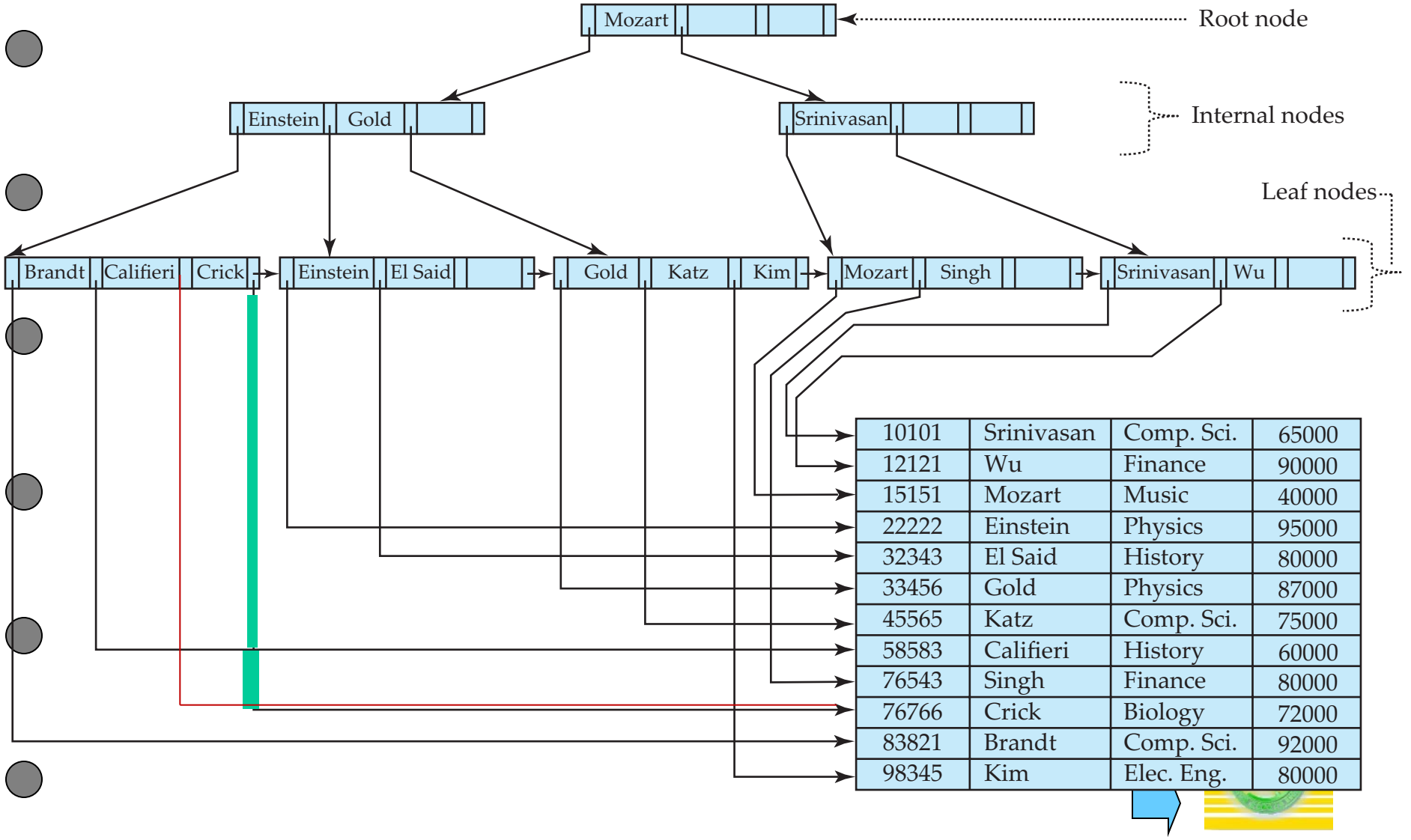
Non-Leaf Nodes in B+-Trees

Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with m pointers:

- All the search-keys in the subtree to which P_1 points are less than K_1
- For $2 \le i \le n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
- All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}

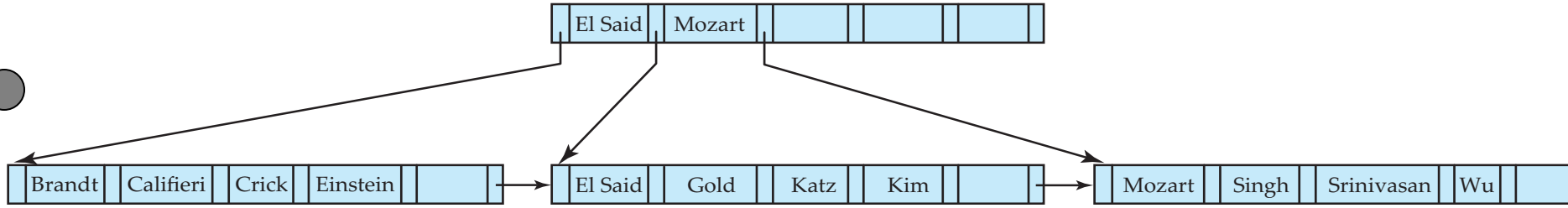
General structure:

Example of B+-tree



Example of B+-tree

B$^+$-tree for *instructor* file (n = 6):

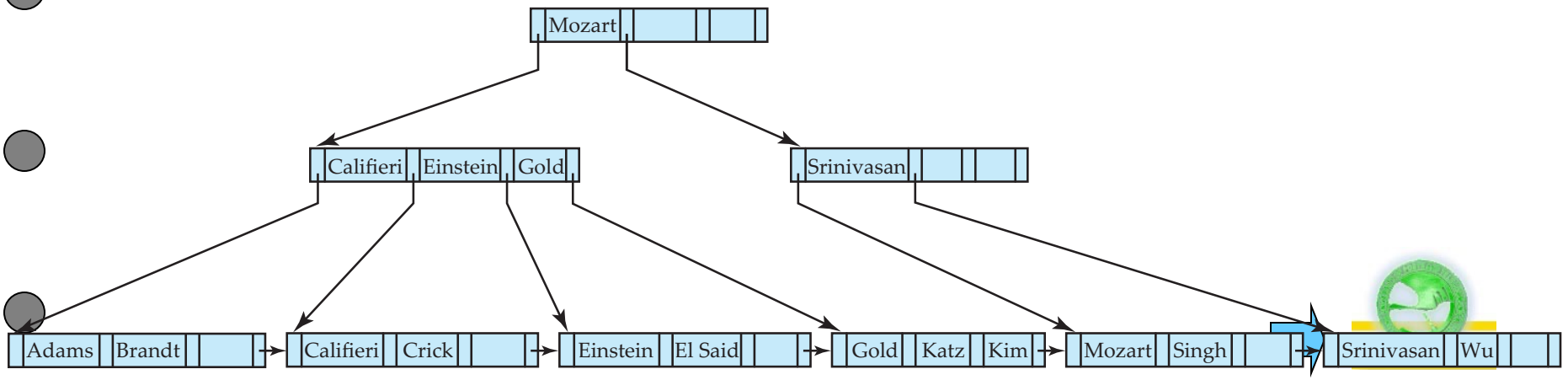


- Leaf nodes must have between 3 and 5 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil (n/2 \rceil$ and n with $n = 6$).
- Root must have at least 2 children.

Queries on B+-Trees

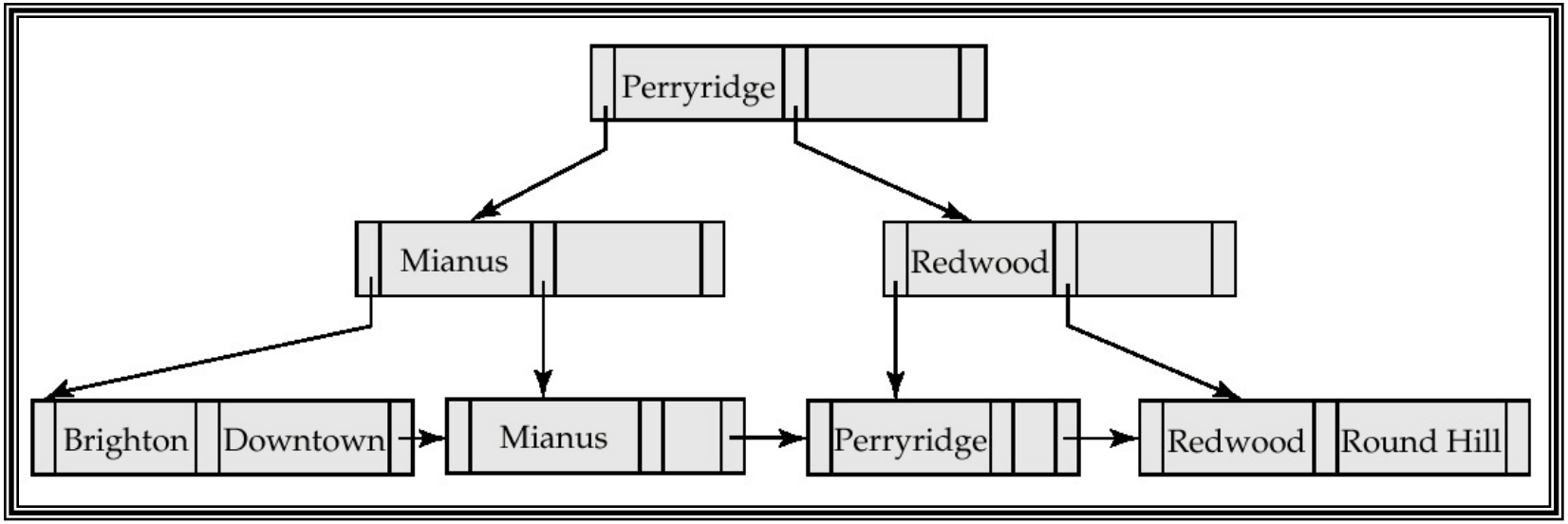
function *find*(*v*)

1. *C=root*
2. **while** (C is not a leaf node)
   1. Let *i* be least number s.t. $V \leq K_i$.
   2. **if** there is no such number *i then*
   3. Set *C = last non-null pointer in C*
   4. **else if** (v = C.K_i) Set C = P_{i+1}
   5. **else set** C = C.P_i
3. **if** for some *i*, $K_i = V$ **then** return C.P_i
4. **else** return null /* no record with search-key value *v* exists. */



Queries on B+-Trees

- If there are K search-key values in the file, the height of the tree is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
- A node is generally the same size as a disk block, typically 4 kilobytes
  - and n is typically around 100 (*40 bytes per index entry*).
- With 1 million search key values and $n = 100$
  - at most $log_{50}(1{,}000{,}000) = 4$ nodes are accessed in a lookup traversal from root to leaf.
- Contrast this with a balanced binary tree with 1 million search key values — around 20 nodes are accessed in a lookup
  - above difference is significant since every node access may need a disk I/O, costing around 20 milliseconds


Perryridge
Mianus
Redwood
Brighton
Downtown
Mianus
Perryridge
Redwood
Round Hill

Hash File Organization Example

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7

Hash file organization of *instructor* file, using *dept_name* as key

Hash Functions

- Typical hash functions perform computation on the internal binary representation of the search-key.
- An ideal hash function is **uniform**i.e., each bucket is assigned the same number of search-key values from the set of *all* possible values.
- Ideal hash function is **random**

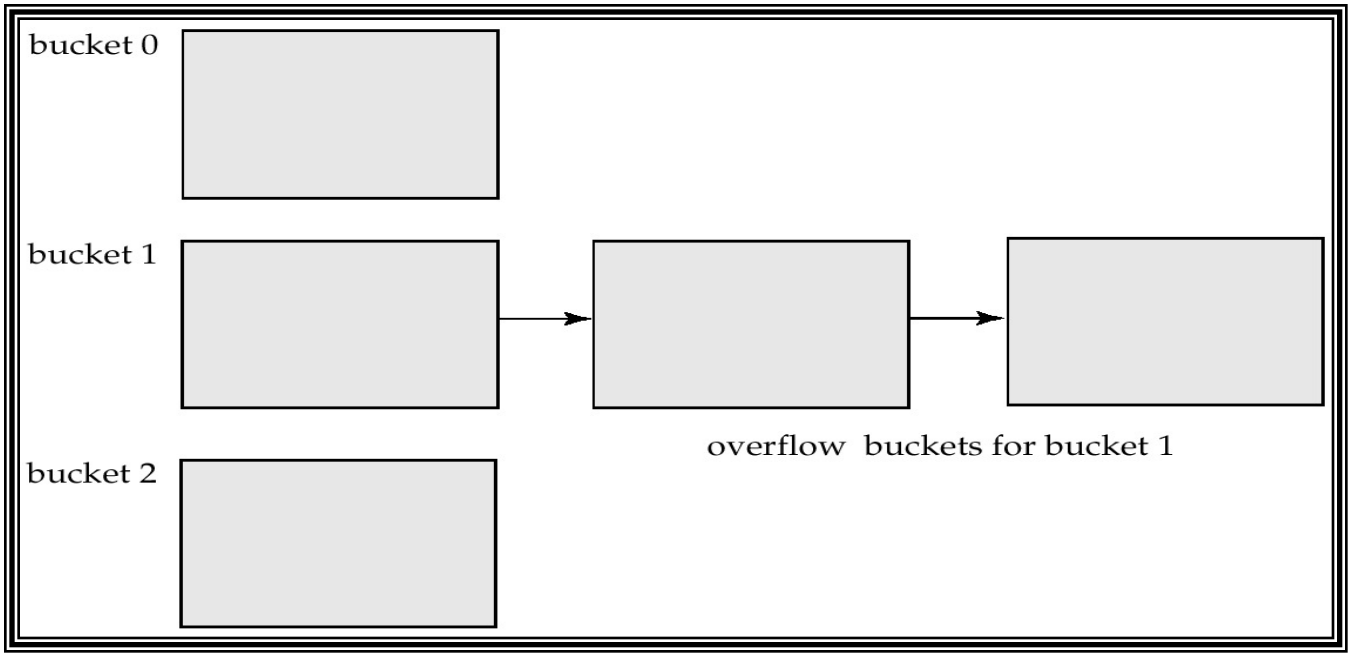
  so each bucket will have the same number of records assigned to it irrespective of the *actual distribution* of search-key values in the file.

Handling of Bucket Overflows(1)

- Bucket overflow can occur because of
  - Insufficient buckets
  - Skew in distribution of records.
    - multiple records have same search-key value;
    - chosen hash function produces non-uniform distribution of key values;
- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using *overflow buckets*.

Handling of Bucket Overflows(2)

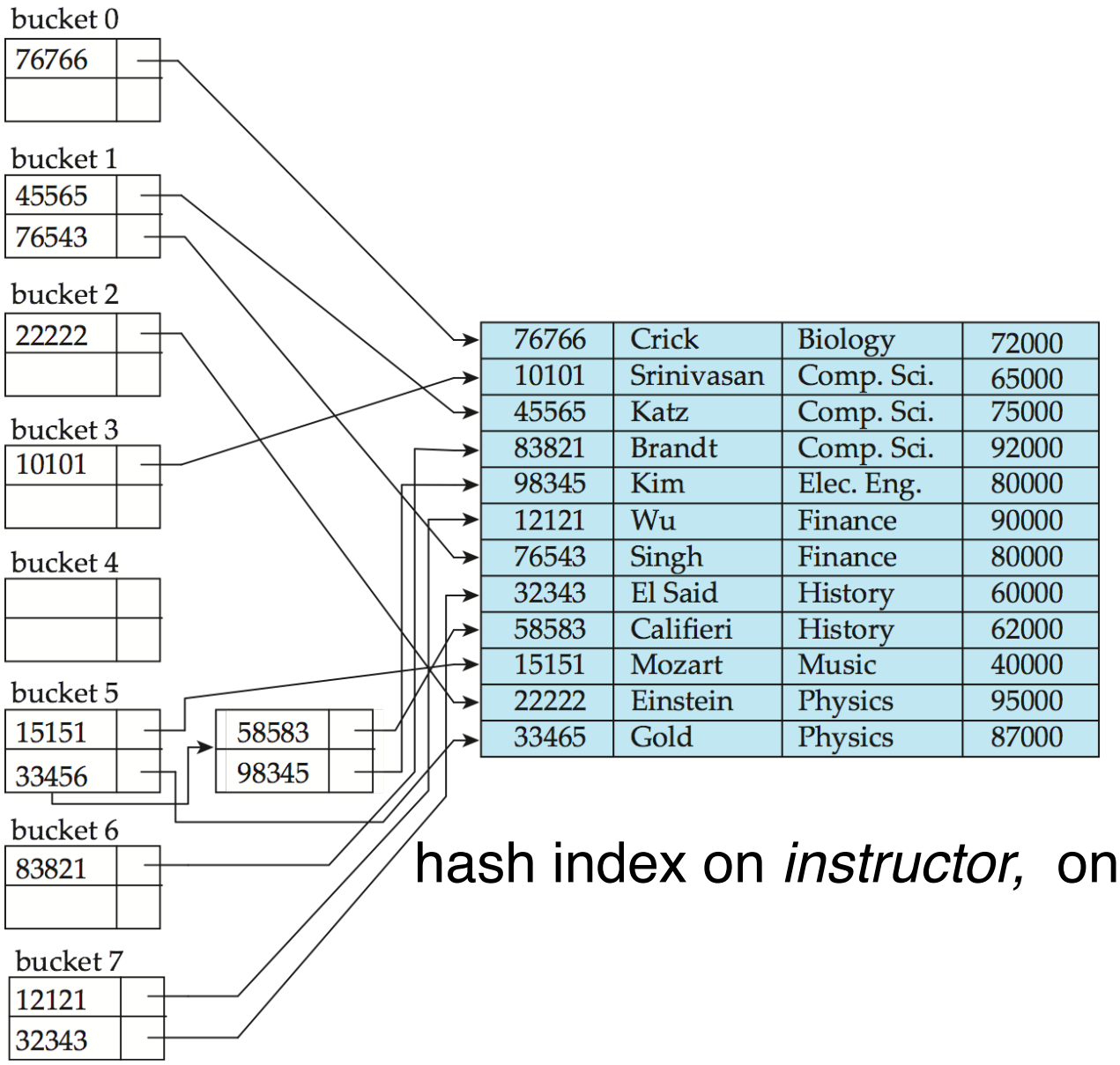
- Overflow chaining/ closed hashing – the overflow buckets of a given bucket are chained together in a linked list.
- An alternative, called open hashing, which does not use overflow buckets, is not suitable for database applications.



Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation.
- A **hash index** organizes the search keys, with their associated record pointers, into a hash file structure.
- Strictly speaking, hash indices are always secondary indices
  - if the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary.
  - However, we use the term hash index to refer to both secondary index structures and hash organized files.

Hash Indices Example



hash index on *instructor,* on attribute *ID*

Deficiencies of Static Hashing

■ In static hashing, function *h* maps search-key values to a fixed set of *B* of bucket addresses.

- Databases grow with time. If initial number of buckets is too small, performance will degrade due to too much overflows.
- If file size at some point in the future is anticipated and number of buckets allocated accordingly, significant amount of space will be wasted initially.
- If database shrinks, again space will be wasted.
- One option is periodic re-organization of the file with a new hash function, but it is very expensive.

- PostgreSQL supports hash indices, but discourages use due to poor performance

- Oracle supports static hash organization, but not hash indices

- SQLServer supports only B^+-trees

Index Definition in SQL

■ Create an index

create [unique] index <index-name> **on** <relation-name>
<attribute-list>)

E.g.: **create index** *b-index* **on** *branch(branch-name)*

- create cluster index

  create noncluster index

  create unique index

■ To drop an index

drop index <index-name>

Multiple-Key Access

- Use multiple single-key indices for certain types of queries.

  select *account-number*

  from *account*

  where *branch-name* = "Perryridge" **and** *balance* =1000

- Possible strategies for processing query using indices on single attributes:

  1. Use index on *branch-name* to find branch with name of Perryridge; test *balance = 1000*.
  2. Use index on *balance* to find accounts with balances of $1000; test *branch-name* = "Perryridge".
  3. Use *branch-name* index to find pointers to all records pertaining to the Perryridge branch. Similarly use index on *balance*. Take intersection of both sets of pointers obtained.

Indices on Multiple Attributes

Suppose we have an index on combined search-key (*branch-name, balance*).

- With the **where** clause
  where *branch-name* = "Perryridge" **and** *balance* = 1000
  the index on the combined search-key will fetch only records that satisfy both conditions.
- Using separate indices in less efficient — we may fetch many records (or pointers) that satisfy only one of the conditions.
- Can also efficiently handle
  where *branch-name* = "Perryridge" **and** *balance* < 1000
- But cannot efficiently handle
  where *branch-name* < "Perryridge" **and** *balance* = 1000
  May fetch many records that satisfy the first but not the second condition.

左前缀原则

在使用复合索引时，查询条件必须从索引的最左列开始匹配，且不能跳过中间的列，才能充分利用索引。

1.从最左列开始：查询条件必须包含复合索引的第一列。例如，对于复合索引 (a, b, c)，查询条件中必须包含 a 列的条件，才能使用该索引。

2.连续使用：可以只使用索引的前几列，但不能跳过中间的列。以下查询条件可以使用索引：

WHERE a = 1 AND b = 2 AND c = 3（完全使用索引）

WHERE a = 1 AND b = 2（使用 a、b 列）

WHERE a = 1（只使用 a 列）

左前缀原则

3. 范围查询后的列失效：某一列使用范围查询（如 >、<、BETWEEN 等）后，其右边的列无法使用索引。
例如，对于复合索引 (a, b, c)，查询条件 WHERE a > 1 AND b = 2 只能使用 a 列，b 列无法使用索引，不能有效使用。

4. 如果查询条件不包含复合索引的最左列，或者跳过了中间的列，索引将无法被有效使用。例如，
WHERE b = 2（缺少最左列 ,无法使用索引）
WHERE b = 2 AND c = 3（缺少最左列，无法使用索引)
WHERE a = 1 AND c = 3 AND b > 2（b 列范围查询后 c 失效，不能有效使用）

左前缀原则

复合索引在存储时按照字段定义的顺序构建多级排序，先按第一列排序，第一列相同再按第二列排序，以此类推。跳过前面的列就相当于在一个无序的数据集中查找，无法利用索引的有序性。

左前缀原则

Suppose we have an index on combined search-key

(*dept_name, salary*).

- With the **where** clause

  where *dept_name* = “Finance” **and** *salary* = 80000

  the index on (*dept_name, salary*) can be used to fetch only records that satisfy both conditions.

- Can also efficiently handle

  where *dept_name* = “Finance” **and** *salary* < 80000

- Can also efficiently handle

  where *dept_name* = “Finance”

左前缀原则

- But cannot efficiently handle

1.

where *dept_name* < “Finance” **and** *salary* = 80000

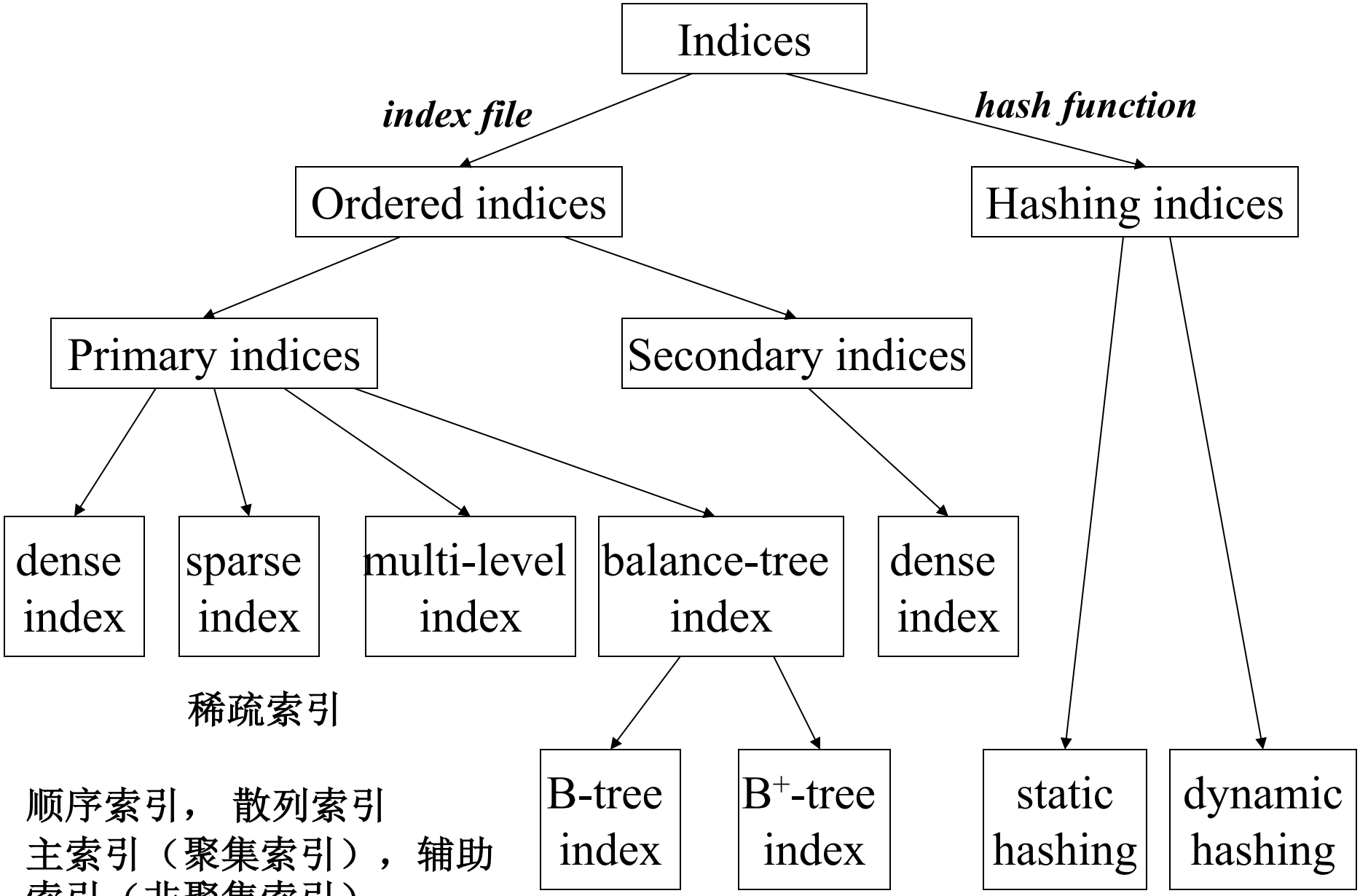
– may fetch many records that satisfy the first but not the second condition

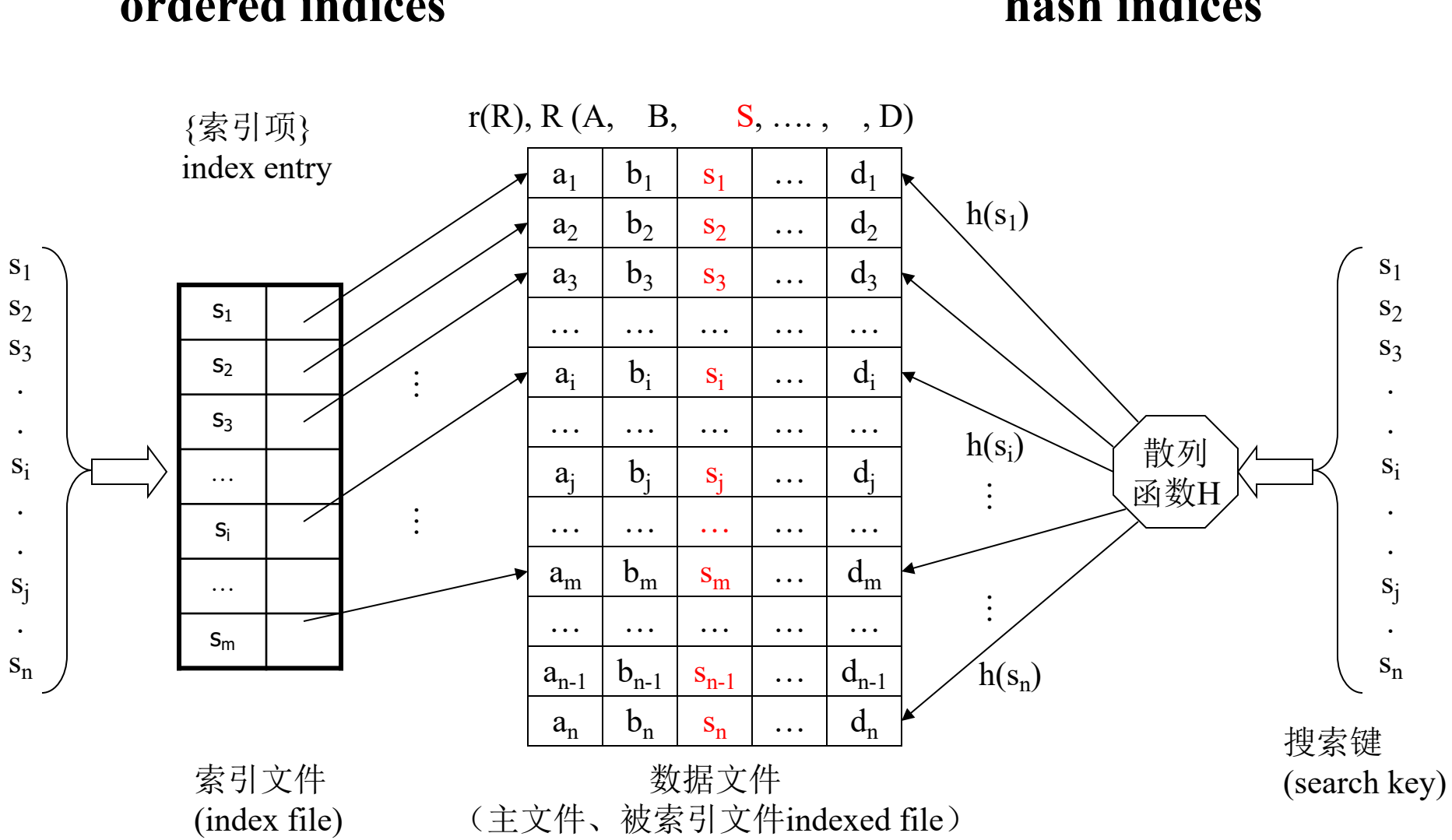
– **the index is not used efficiently**

2.

where *salary* = 80000

– **the index is not used for query**


Indices
index file
hash function
Ordered indices
Hashing indices
Primary indices
Secondary indices
dense index
sparse index
multi-level index
balance-tree index
dense index
稀疏索引
B-tree index
B+-tree index
static hashing
dynamic hashing
顺序索引， 散列索引
主索引（聚集索引），辅助索引（非聚集索引）


ordered indices
hash indices
{索引项}
index entry
r(R), R (A, B, S, …. , , D)
h(s1)
h(si)
h(sn)
散列
函数H
索引文件
(index file)
数据文件
（主文件、被索引文件indexed file）
搜索键
(search key)

索引及其分类

Summary

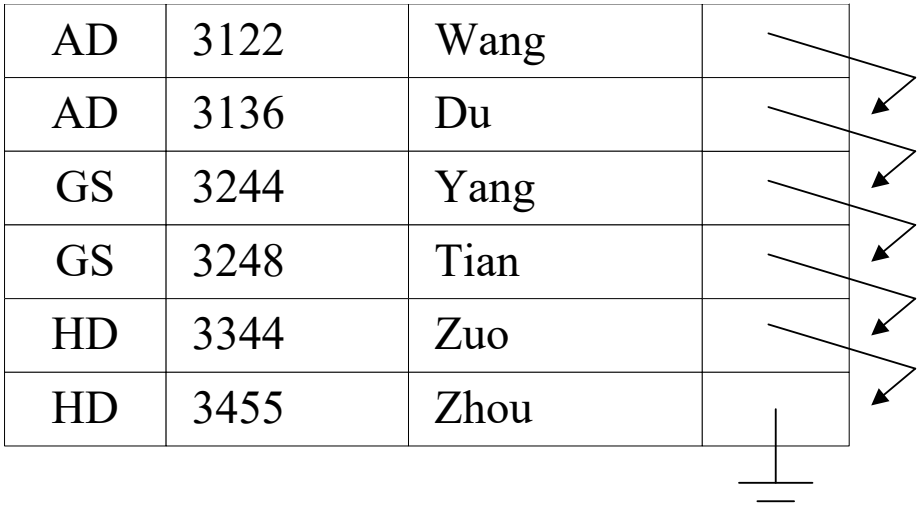
- Ordered index
  - Primary index
    - Clustering index
  - Secondary index
  - Dense index
  - Sparse index
- B+-Tree index, B- Tree index
- Hash index
- Index Definition

Give the data file ***teacher***(s_dept, student_ID, student_name) as shown below, which is organized as a sequential file, taking the attributes s_*dept* as the search key,

(1) *D*efine a *dense* and *clustering* index for the indexed file ***teacher*** on attribute student_name. It is required that the index file and index entries in the index file should be figured out.

(2) If a tuple (BD, 3188, Ren) is inserted into the indexed file, depict the indexed file and the index file.

(s-dept, student_ID, student_name)



AD	3122	Wang	
AD	3136	Du	
GS	3244	Yang	
GS	3248	Tian	
HD	3344	Zuo	
HD	3455	Zhou	

Consider the following tables in the database ***University***:

Student(*sid*, *name*, *department*, *age*)

Department(*dname*, *building*, *budget*)

(1) Consider the following SQL query. In addition to the existing primary indices on the primary keys of the tables, on which attributes in the tables the indices can be further defined to speed up the query?

```
select  department, sum(sid)
from  Student as A, Department as B
where  building='T3' and age>20 and A.department= B.dname
group by department
```

(2) For the following query

```
update Student
set  age=age+2
where sid between 211301 and 211318
```

A nonclustered index has been defined on the attribute *age*.

Does this index speed up or slow down the operation, and why?